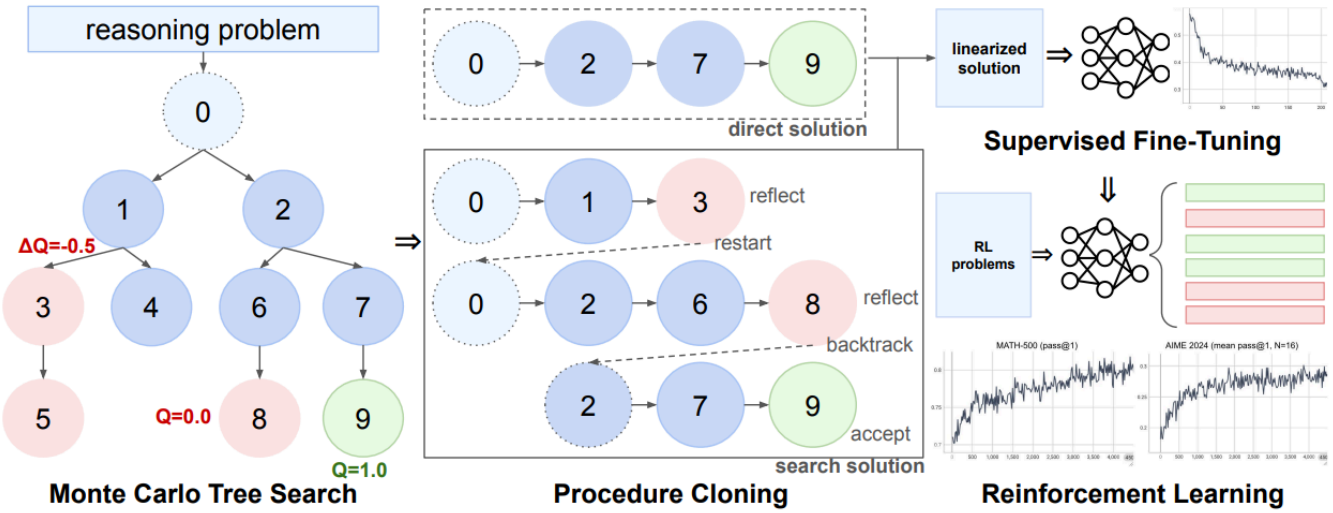
ASTRO: Teaching Language Models to Reason by Reflecting and Backtracking In-Context

> #Finetune #Dataset #RL #CoT



- What problems does this framework solve?
  - A lot of LLMs, currently, have already gained the ability to create complex CoT and solve hard tasks.
  - However, they still lack the ability to correct their errors. That is to say, if one step in their CoT is wrong, then the rest of the CoT, including the final answer, are all wrong.
  - Thus, a framework is needed to teach the model to get rid of the wrong step, drawback to the last correct step, and start a new route.
- What does this framework do?
  - This framework takes advantage of the Monte Carlo Tree algorithm to teach the model to find out and reflect when they have trouble.

  1. Monte Carlo Tree Search Method to Generate the Dataset.
     - We use PUCT to balance the exploration (encourage the model to explore actions/states that they haven't seen) and exploitation (use historically proven action/states that can lead to success)

       $$S_{t+1}^* = \text{argmax}_{(S_{t+1}=S_t \to a_i)} \left[Q(S_t, a_i) + c_{puct} \cdot \Pi_{LM}(a_i|S_t) \frac{\sqrt{N(S_t)}}{1+N(S_t,a_i)} \right]$$

       - $Q(S_t, a_i)$: is the "historical proof": this score is maintained by history. If this state was visited, and the result was good, then the score of this route would be high.
       - $\frac{\sqrt{N(S_t)}}{1+N(S_t,a_i)}$: is the "exploration score". If a route is hardly visited (N is small), then this term will be large.
       - Π_{LM}: what does the model think of the new way?
       - We select the route with max score to go along with.
     - Expand the tree: go along the selected route until the end. For the selected nodes, we

sample k actions, and for each action, we run M simulations and collect the results after being evaluated. Then, calculate the average score of this path:

$R(S_{t+1}) = \frac{1}{M}\sum_{j\in[1\ldots M]} V(\Pi_{LM,j}(S_{t+1})),\ V(\cdot)\in 0,1$

- Backpropagation: we pass the reward score back to update the Q value of the ancestors (using average). We also update the visited-counter N.

  $N(S_t) = N(S_t) + 1$

  $Q(S_t, a) = \frac{\sum_{i=1}^{K} Q(S_{t+1},a_i)\cdot N(S_{t+1},a_i) + R(S_{t+1})}{\sum_{i=1}^{K} N(S_{t+1},a_i)+1}$

2. Linearize the route and construct NL description.
   - After collecting the routes (trajectories), we start from the root and record a route to the leaf with the correct answer. Along the route, we insert steps of the wrong branches (backtrack).

     `root-> step 1 -> wrong answer -> step 1 -> step 2 -> wrong answer -> step 2 -> step 3-> correct answer`.

   - Translate the tree path to NL with reflection tokens (e.g., *"But wait, are we solving the problem correctly so far? Hmm... I'm not sure..."*) and backtrack tokens (e.g., *"Let's go back to the last step."*) inserted.
3. Finetune and RL
   - With the dataset we constructed in b, we run SFT to teach the model to *reflect-backtrack style CoT*.
   - To make the model master a certain type of task (e.g., *math problems*), use GRPO to further finetune the model.

- Experiment Results
  - Significant improvements: The ASTRO-RL model achieved 81.8% on MATH-500 (a 16.0% increase), 64.4% on AMC 2023 (a 26.9% increase), and 30.0% on AIME 2024 (a 20.0% increase).
  - Emergence of Search Behavior: As RL training progressed, the model generated longer CoT sequences and exhibited a significant increase in backtracks.
  - Backtracks positively correlate with accuracy: The number of backtracks during testing showed a strong positive correlation (Pearson coefficient > 0.8) with final problem-solving accuracy, proving the model genuinely solves problems through “backtracking” rather than generating meaningless output.
  - Outperforms baselines without search priors: Comparative experiments show that without the search prior injected during the SFT phase (i.e., using standard CoT for RL), the model's performance ceiling and convergence speed are significantly inferior to ASTRO.

- My Thought
  - The same methodology can be used for tool-call chain optimization.
  - We use Monte Carlo Tree Search to let the model explore the chain of tool calls, and evaluate the result of the chain. Following the same method (expansion, backpropagation, linearization, and translate to NL), we collect backtrack-included tool-call CoT records.
  - Then, SFT+GRPO to make the model get used to specific types of tasks.
  - However, the process can be very costly as the tree can be very wide, and the tool calls may need more time. Thus, it is necessary to decrease the # of sampled actions (k).

- Work Cited
    - Paper
    - NotebookLM
    - Chat History